

Indice

Easyfatt Sync — Documentazione tecnica sviluppatori

1. Introduzione progetto

Cos'è Easyfatt Sync

Obiettivo software

Stack tecnologico

2. Architettura applicazione

Diagramma logico

Main process

Renderer process

Preload

IPC (canali principali)

electron-store

Scheduler

Sync engine

Diff engine (nuovo in 26.0.0)

Google OAuth

Updater

Backup system

Support system

3. Struttura progetto

File principali (ruolo)

4. Configurazione sviluppo

Prerequisiti

Installazione

Configurazione credenziali OAuth

Avvio in sviluppo

Script build

Variabili ambiente

5. Google OAuth setup

Progetto Google Cloud (Aven Labs)

OAuth Desktop App

Scopes

File credenziali

Token locali

Sicurezza

6. Sistema sincronizzazione

Flusso dati

Watcher file

Sync programmata

Gestione Excel

Google Sheets

Progress UI

Gestione errori

7. Sistema aggiornamenti automatici

Componenti

Comportamento app

Release workflow (sintesi)

Versioning (calendar-based dal 2026)

8. Sistema supporto

Architettura

Payload (campi principali)

Backend di esempio

Email cliente

9. Backup e ripristino

Formato file JSON

Struttura payload

Restore

Backup automatici

10. Sicurezza

Electron hardening

Preload sicuro

Token e credenziali

Repository sicurezza (cose da non committare)

CI secret handling

API backend

Buone pratiche release

11. Workflow release

GitHub Actions (release ufficiale)

Checklist pre-tag

Comandi tag/release

Release manuale (fallback)

Publish locale con token (raro)

Compatibilità aggiornamenti

12. TODO / roadmap

13. Troubleshooting tecnico

OAuth

Sync

Updater

Build

Permessi file

Token Google in backup

Contatti tecnici

Easyfatt Sync — Documentazione tecnica sviluppatori

Versione documentazione allineata a Easyfatt Sync **26.0.0** (ciclo 2026, stack Electron 42, backup v1.0).

Documentazione di riferimento per sviluppo, manutenzione, build e release dell'applicazione desktop **Easyfatt Sync** (Aven Labs).

1. Introduzione progetto

Cos'è Easyfatt Sync

Easyfatt Sync è un'applicazione desktop che automatizza l'aggiornamento di un foglio **Google Sheets** a partire dal file **Excel** esportato da **Easyfatt** (tipicamente elenco clienti).

Il cliente finale non configura Google Cloud: usa credenziali OAuth **centralizzate Aven Labs** (`oauth_credentials.json` nel pacchetto) e salva solo il **token personale** in locale.

Obiettivo software

Obiettivo	Descrizione
Affidabilità	Sync idempotente (clear + rewrite foglio), retry lettura Excel su file bloccato
Semplicità	UI a impostazioni, senza terminale

Obiettivo	Descrizione
Automazione	Watch file, cron, promemoria, backup automatici
Manutenibilità	Moduli Node separati, IPC tipizzati via preload
Distribuzione	Installer Windows (NSIS) e DMG macOS, update via GitHub Releases

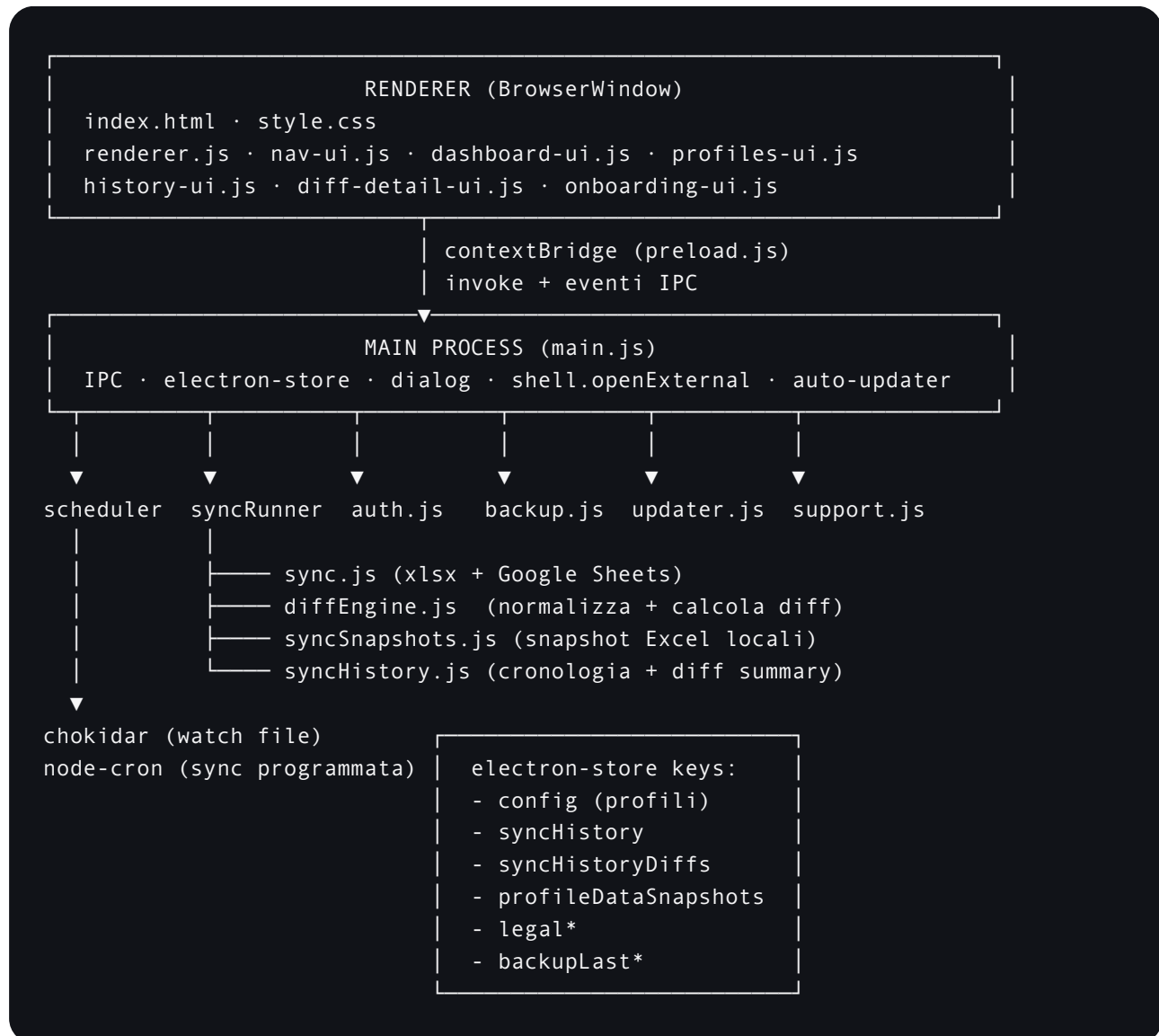
Stack tecnologico

Layer	Tecnologia
Desktop shell	Electron 42.x
Runtime	Node.js (bundled con Electron)
UI	HTML / CSS / JavaScript vanilla (<code>renderer /</code>)
Persistenza config	<code>electron-store</code>
Excel	<code>xlsx</code> (lettura buffer async)
Google	<code>googleapis</code> + OAuth 2.0
File watch	<code>chokidar</code>
Scheduler	<code>node-cron</code>
Build	<code>electron-builder</code>
Auto-update	<code>electron-updater</code> + <code>electron-log</code>
HTTP supporto	<code>fetch</code> nativo verso API Aven Labs

Piattaforme target: Windows 10/11 (x64), macOS (ARM64 e Intel).

2. Architettura applicazione

Diagramma logico



Main process

- Punto di ingresso: `main.js`.
- Crea `BrowserWindow`, registra handler `ipcMain`, orchestra moduli.
- **Non** espone Node al renderer (`nodeIntegration: false`, `contextIsolation: true`).
- All'avvio: carica config, `restartScheduler`, login item, check update (solo packaged).
- All'uscita: `stopScheduler()` su `before-quit`.

Renderer process

- Carica `renderer/index.html` + più moduli JS coordinati da `renderer.js`.
- Interagisce solo con `window.easyfattSync` (API preload).
- Modulo `nav-ui.js`: layout a 3 colonne (sidebar · main · side panel), routing tra viste, persiste vista attiva in `localStorage`.
- Modulo `dashboard-ui.js`: dashboard salute, ultima sync, prossima sync, contatori giornalieri.
- Modulo `profiles-ui.js`: gestione profili sync, empty state intelligente (testo cambia da “prima sincronizzazione” a “un’altra sincronizzazione” in base a profili esistenti).
- Modulo `history-ui.js`: cronologia eventi, filtri, mini-badge diff (+ / ~ / -), click-to-detail.
- Modulo `diff-detail-ui.js`: vista *Dettaglio sincronizzazione* stile GitHub diff.
- Modulo `onboarding-ui.js`: wizard 6-step.
- Gestisce inoltre: form impostazioni, attività recenti (max 50), tema chiaro/scuro persistito, legal gate, supporto, backup UI, aggiornamenti.

Preload

File: `preload.js`.

Espone un sottoinsieme sicuro di operazioni tramite

```
contextBridge.exposeInMainWorld("easyfattSync", { ... })
```

Pattern:

- Operazioni request/response → `ipcRenderer.invoke(channel, ...args)`.
- Eventi main → renderer → `ipcRenderer.on("log" | "config-updated" | "update-state" | ...)`.

IPC (canali principali)

Canale	Direzione	Funzione
<code>get-config</code> / <code>save-config</code>	R  M	Lettura/scrittura config
<code>select-excel</code> / <code>select-backup-folder</code>	R → M	Dialog file/cartella
<code>connect-google</code> / <code>logout-google</code> / <code>is-google-authorized</code>	R  M	OAuth

Canale	Direzione	Funzione
<code>sync-now</code>	R → M	Sync manuale
<code>backup-create</code> / <code>backup-preview</code> / <code>backup-restore</code> / <code>get-backup-meta</code>	R ? M	Backup
<code>check-for-updates</code> / <code>download-update</code> / <code>install-update-now</code>	R ? M	Updater
<code>submit-support-request</code>	R → M	Supporto
<code>get-legal-status</code> / <code>accept-legal</code>	R ? M	Privacy/termini
<code>open-external</code>	R → M	Link HTTPS nel browser di sistema
<code>get-sync-history</code> / <code>export-history-report</code>	R ? M	Cronologia eventi
<code>get-history-diff</code>	R ? M	Dettaglio diff di un evento
<code>clear-sync-history</code>	R → M	Reset cronologia + snapshot
<code>log</code>	M → R	Stream log attività
<code>config-updated</code>	M → R	Refresh UI dopo sync/restore
<code>update-state</code> / <code>update-progress</code>	M → R	Stato aggiornamenti

electron-store

- Istanza singleton in `main.js`: `const store = new Store()`.
- Chiavi principali:
 - `config` — oggetto impostazioni utente (vedi `syncState.getDefaultConfig()`), include `syncProfiles[]`.
 - `syncHistory` — array eventi sync (max **500**, ognuno con `diffSummary` inline e flag `hasDiffDetails`).

- `syncHistoryDiffs` — mappa `{ eventId → diffDetails }`, max **50** eventi con dettaglio, max **500 righe** per categoria, valori troncati a **500 caratteri**.
- `profileDataSnapshots` — mappa `{ profileId → { headers, rows[], capturedAt } }`, max **20000 righe** per snapshot.
- `legalAccepted`, `legalAcceptedAt`, `legalVersion`.
- `backupLastCreatedAt`, `backupLastRestoredAt`, `backupLastAutomaticAt`.

Scheduler

Modulo: `scheduler.js`.

Responsabilità:

- **File watcher** (`chokidar`) con `awaitWriteFinish` e `debounce 5s`.
- **Sync programmata** (`node-cron`) su orari `syncTimes`.
- **Promemoria sync mancante** (`reminderTimes`).
- **Backup automatici** (cron giornaliero/settimanale/mensile).

Ogni `restartScheduler` esegue `stopScheduler()` (chiude watcher, ferma cron, cancella timeout) prima di riavviare.

Mutex sync: `syncRunner.runSync` usa flag `syncInProgress` condiviso tra manuale, watch e cron.

Sync engine

Pipeline:

1. `sync.js` — legge Excel (async + retry), prepara matrice valori, chiama Google Sheets API. Restituisce anche `headers` e `normalizedRows` per il diff.
2. `syncRunner.js` — orchestrazione: mutex `syncInProgress`, notifiche, `recordSyncSuccess`.
Post-sync: chiama `computeDiffAndSnapshot()` che (a) recupera snapshot precedente, (b) calcola diff via `diffEngine.calculateDiff`, (c) salva il nuovo snapshot, (d) passa `diffSummary` + `diffDetails` a `recordSyncEvent`.
3. `diffEngine.js` — `normalizeRows`, `getRowKey` (primary key configurabile / candidate Codice / Email / hash MD5 fallback), `calculateDiff` con limite `MAX_DIFF_TOTAL` per file enormi.
4. `syncSnapshots.js` — persistenza snapshot per profilo, `pruneOrphanSnapshots` al salvataggio config, `clearAllSnapshots` al reset cronologia.
5. `syncHistory.js` — `recordSyncEvent` con `diffSummary` inline; `syncHistoryDiffs` separati con pruning (`MAX_DIFF_EVENTS=50`, `MAX_DIFF_ROWS_DETAIL=500`).

6. `sheetRange.js` — range dinamico colonne (non più `A:Z` fisso).
7. `errors.js` — messaggi client-friendly.

Diff engine (nuovo in 26.0.0)

```
runSync(profile)
├─ sync.js → { headers, normalizedRows, success, rowCount, durationMs }
├─ computeDiffAndSnapshot(profile, headers, rows)
│   ├─ previous = syncSnapshots.getSnapshot(profile.id)
│   ├─ diff      = diffEngine.calculateDiff(previous, current, profile)
│   ├─ syncSnapshots.setSnapshot(profile.id, { headers, rows, capturedAt })
│   └─ return { diffSummary, diffDetails }
├─ syncHistory.recordSyncEvent({ ..., diffSummary, diffDetails })
│   ├─ store.syncHistory.push(event con diffSummary inline)
│   ├─ if diffDetails: syncHistoryDiffs[event.id] = sanitize(diffDetails)
│   └─ pruneDiffs() — mantiene ultimi 50 eventi con dettaglio
```

Privacy: i `diffDetails` (contengono dati clienti) non vengono mai:

- inclusi nei backup di default;
- inviati al supporto via `diagnostics.js` (deletion esplicita);
- esposti via altri canali oltre alla modale locale `diff-detail-ui.js`.

Google OAuth

Modulo: `auth.js`.

- Credenziali app: `oauth_credentials.json` (root progetto, **non** in git).
- Token utente: `%APPDATA%/EasyfattSync/token.json` (Windows) o `~/EasyfattSync/token.json` (macOS/Linux).
- Flusso: server HTTP locale temporaneo su `127.0.0.1`, browser esterno, callback, salvataggio token.
- Cache client OAuth + listener `tokens` per persistere refresh.

Updater

Modulo: `updater.js`.

- Attivo solo se `app.isPackaged`.

- `autoDownload: false`, `autoInstallOnAppQuit: false` — l'utente conferma download e install.
- Publish configurato verso GitHub (`package.json` → `build.publish`).
- Eventi verso renderer su canale `update-state` .

Backup system

Modulo: `backup.js` .

- Export JSON impostazioni + stato legale + token Google opzionale.
- Restore con validazione `backupVersion` .
- Rotazione backup automatici per cartella e retention N.
- Lock `backupInProgress` per evitare backup concorrenti.

Support system

Moduli: `support.js` , `supportConstants.js` , `supportIssueTypes.js` .

- POST JSON a `https://aven-labs.com/api/support/easyfatt-sync` .
- Timeout 30s, validazione lato client e server (esempio Next.js in `backend-examples/`).

3. Struttura progetto

```
easyfatt-sync-app/
├─ oauth_credentials.example.json # Template OAuth (committato, senza secret)
├─ oauth_credentials.json         # Credenziali reali (gitignored, solo locale/
build)
├─ main.js                       # Entry Electron, IPC, lifecycle
├─ preload.js                   # Bridge sicuro renderer ↔ main
├─ auth.js                      # OAuth Google, token file
├─ sync.js                      # Excel → Google Sheets (ritorna headers +
normalizedRows)
├─ syncRunner.js                # Mutex sync, notifiche, post-sync diff & snapshot
├─ syncState.js                 # Profili sync, merge, migrazione legacy
├─ syncHistory.js               # Cronologia eventi (max 500) + diffSummary inline +
diff details separati
├─ syncSnapshots.js            # Snapshot Excel per profilo (electron-store, pruning
orfani)
├─ diffEngine.js                # Normalizzazione, row key, calculateDiff stile
```

```

GitHub
├─ healthStatus.js          # Calcolo dashboard salute
├─ excelUtils.js           # Validazione/preview Excel, retry lettura
├─ columnMapping.js        # Mapping colonne per profilo
├─ diagnostics.js          # Report diagnostico supporto (sanitizzato, no diff
details)
├─ appConstants.js         # Costanti prodotto / rebranding futuro
├─ scheduler.js            # Watch, cron, backup automatico, multi-profilo
├─ backup.js               # Backup/restore JSON (no diff/snapshot per privacy)
├─ updater.js              # electron-updater
├─ support.js              # Invio richiesta supporto
├─ supportConstants.js     # URL API, email
├─ supportIssueTypes.js    # Etichette tipo problema
├─ notifications.js        # Notifiche desktop native
├─ loginSettings.js        # Avvio con il sistema (openAtLogin)
├─ errors.js               # Messaggi errore UI
├─ sheetRange.js           # Helper range Sheets
├─ legalConstants.js       # Versione legale, URL policy
├─ package.json            # Scripts, electron-builder, publish (version:
26.0.0)
├─ UPDATES.md              # Workflow release operativo + secret GitHub Actions
├─ VERSIONING.md           # Strategia calendar versioning (MAJOR = anno)
├─ RELEASE_NOTES.md        # Note release 26.0.0
├─ SECURITY.md              # Policy vulnerabilità + file da non committare
├─ .github/
│   └─ workflows/
│       └─ release.yaml     # CI build Win+Mac su tag v*.*.*
├─ assets/
│   └─ icon.png            # Icona app / build
├─ renderer/
│   └─ index.html
│   └─ style.css
│   └─ renderer.js         # Entry UI, theme, version footer
│   └─ nav-ui.js           # Sidebar, viste, header CTA, save bar conditional
│   └─ dashboard-ui.js     # Dashboard salute
│   └─ profiles-ui.js      # Gestione profili + empty state dinamico
│   └─ history-ui.js       # Cronologia, filtri, click-to-detail
│   └─ diff-detail-ui.js   # Modale Dettaglio sync (tab + GitHub diff)
│   └─ onboarding-ui.js    # Wizard 6-step
├─ legal/
│   └─ privacy-easyfatt-sync.md
│   └─ terms-easyfatt-sync.md
├─ backend-examples/       # API supporto (Next.js + Resend)
└─ docs/                   # Questa documentazione

```

File principali (ruolo)

File	Ruolo
<code>main.js</code>	Window, IPC, collegamento moduli, <code>restartScheduler</code> , prune snapshot
<code>preload.js</code>	API <code>window.easyfattSync</code> (incluso <code>getHistoryDiff</code> , <code>clearSyncHistory</code>)
<code>auth.js</code>	OAuth loopback, read/write token, cache client
<code>sync.js</code>	Lettura XLSX, clear/update Sheets, ritorna <code>normalizedRows</code>
<code>syncRunner.js</code>	Mutex sync, notifiche, diff & snapshot post-sync
<code>diffEngine.js</code>	Normalizza righe, getRowKey, calculateDiff (added/modified/removed)
<code>syncSnapshots.js</code>	Snapshot Excel per profilo, prune orfani, clear all
<code>syncHistory.js</code>	Eventi sync + <code>diffSummary</code> inline + <code>syncHistoryDiffs</code> pruned
<code>scheduler.js</code>	Automazioni temporizzate, watch file, multi-profilo
<code>backup.js</code>	Payload backup, restore, cleanup (no diff/snapshot)
<code>updater.js</code>	Check/download/install release GitHub
<code>support.js</code>	Validazione form + fetch API
<code>renderer/nav-ui.js</code>	Sidebar, viste, distribuzione accordion → view
<code>renderer/diff-detail-ui.js</code>	Modale Dettaglio sync, render GitHub diff
<code>renderer/renderer.js</code>	Logica UI, attività, form, modali, theme

4. Configurazione sviluppo

Prerequisiti

- **Node.js** 18+ (consigliato LTS allineato a Electron 42).
- **npm** 9+.
- Account Google Cloud (solo per generare `oauth_credentials.json` di test).
- Per build macOS: macOS + Xcode CLI tools.
- Per build Windows: Windows (o CI) per NSIS.

Installazione

```
git clone <repository-url>
cd easyfatt-sync-app
npm install
```

Configurazione credenziali OAuth

Il file reale `oauth_credentials.json` **non deve essere committato** nel repository pubblico.

Setup locale (sviluppo e build):

1. Copia il template:

```
cp oauth_credentials.example.json oauth_credentials.json
```

2. Apri `oauth_credentials.json` e inserisci le credenziali del **OAuth Client Desktop App** del progetto Google Cloud ufficiale **Aven Labs** (`client_id` , `client_secret` , ecc.).
3. Non committare il file reale: è già in `.gitignore` .

Build produzione (installer Windows/macOS):

- Prima di `npm run dist:win` o `npm run dist:mac` , assicurati che `oauth_credentials.json` esista **nella root del progetto** sulla macchina di build.
- electron-builder includerà il file nel pacchetto installato; ogni cliente finale **non** crea un progetto Google Cloud.

Token per utente (`token.json`):

- Generato **solo dopo** che l'utente clicca "Collega Google" nell'app installata.
- Salvato in locale per macchina, **mai** nel repository:
 - macOS/Linux: `~/EasyfattSync/token.json`
 - Windows: `%APPDATA%/EasyfattSync/token.json`

Se `oauth_credentials.json` manca, l'app mostra:

Credenziali Google OAuth mancanti. Configura `oauth_credentials.json`.

Dettagli Google Cloud: [§5 Google OAuth setup](#). Sicurezza repository: `SECURITY.md`.

Avvio in sviluppo

```
npm run dev
```

Equivalente a `electron` — **non** abilita auto-updater reale (messaggio dev in UI aggiornamenti).

Script build

Script	Output
<code>npm run dist:win</code>	Installer NSIS x64 in <code>dist/</code>
<code>npm run dist:mac</code>	DMG ARM64 (Apple Silicon), senza firma/notarizzazione
<code>npm run dist:mac:intel</code>	DMG x64 (Intel), senza firma/notarizzazione
<code>npm run icons:mac</code>	Genera <code>assets/icon.icns</code>
<code>npm run check</code>	Controllo salute progetto (pre-build / pre-push)

Guide build: [build-windows.md](#) · [build-macos.md](#) · [notarizzazione futura](#).

Configurazione builder in `package.json` → sezione `"build"` :

- `appId` : `com.avenlabs.easyfattsync`
- `productName` : `Easyfatt Sync`
- Windows: target **NSIS** (`oneClick: false` , scelta cartella, shortcut Desktop/Start, `assets/icon.ico`)
- macOS: **DMG** con `assets/icon.icns` , `hardenedRuntime: false` (test); notarizzazione documentata in [macos-notarization-future.md](#)
- `publish` : GitHub `DanielMatei0/easyfatt-sync`
- `oauth_credentials.json` : incluso nel pacchetto se presente in root al build; **mai** in Git (`oauth_credentials.example.json` resta nel repo)

Variabili ambiente

Variabile	Uso
<code>GH_TOKEN</code>	Publish release su GitHub da CI o macchina build (electron-builder)
<code>RESEND_API_KEY</code>	Solo backend supporto (Next.js), non nell'app Electron
<code>APPDATA</code> / <code>HOME</code>	Path token utente (lettura in <code>auth.js</code>)

File **non** committare (vedi `.gitignore`):

- `oauth_credentials.json`
- `token.json`
- `.env` / `.env.local`

5. Google OAuth setup

Progetto Google Cloud (Aven Labs)

1. Console [Google Cloud](#).

2. Crea o seleziona un progetto (es. "Aven Labs Easyfatt Sync").
3. Abilita **Google Sheets API**.
4. Crea credenziali **OAuth 2.0 Client ID** di tipo **Desktop app** (consigliato per Electron).

OAuth Desktop App

- Con Desktop app i redirect `http://127.0.0.1:<porta>/callback` sono gestiti senza registrare ogni porta in anticipo (a differenza di alcuni Web client).
- L'app avvia un server locale temporaneo e apre il browser con `shell.openExternal`.

Scopes

```
https://www.googleapis.com/auth/spreadsheets
```

Accesso in lettura/scrittura ai fogli dell'account collegato.

File credenziali

`oauth_credentials.json` (formato Google):

```
{
  "installed": {
    "client_id": "...",
    "client_secret": "...",
    "redirect_uris": ["http://localhost"]
  }
}
```

In produzione il file è **incluso nel pacchetto**; il cliente non crea un progetto Google proprio.

Token locali

- Salvati in `EasyfattSync/token.json` per utente/macchina.
- `access_type: "offline"` + `prompt: "consent"` al primo collegamento per ottenere refresh token.
- Refresh persistito su evento `tokens` del client OAuth.

Sicurezza

Regola	Implementazione
Secret non nel renderer	Solo main process legge credenziali
Token non in backup di default	Checkbox esplicita in UI backup
Revoca	“Disconnetti Google” elimina <code>token.json</code>
Messaggio scadenza	<code>errors.js</code> + refresh fallito → “Ricollega Google”

6. Sistema sincronizzazione

Flusso dati

```
Excel (.xlsx) → xlsx (sheet_to_json) → matrice [header, ...rows]
→ values.clear (range dinamico) → values.update (A1, RAW)
```

- Viene usato il **primo foglio** del workbook Excel (`SheetNames[0]`).
- Il nome foglio Google è `config.sheetName` (default `Clients`).

Watcher file

- Attivo se `watchEnabled && excelPath` .
- `chokidar` con `awaitWriteFinish.stabilityThreshold: 5000` ms.
- Debounce aggiuntivo 5s prima di chiamare `runSync` .
- Skip se sync già in corso o config incompleta (`canRunAutomatedSync`).

Sync programmata

- `scheduleEnabled` + array `syncTimes` (formato `HH:MM`).
- Un job `node-cron` per ogni orario valido.

- Orari non validi ignorati con warning in log.

Gestione Excel

- Lettura **async** (`fs.promises.readFile` + `XLSX.read` `buffer`).
- **Retry** fino a 5 tentativi su `EBUSY` / `EPERM` (file aperto in Easyfatt/Excel).
- Messaggio utente: file in uso → chiudi Easyfatt e riprova.

Google Sheets

- Clear solo sulle colonne effettive (`sheetRange.buildClearRange`).
- Update da `'SheetName'!A1` con tutte le righe.

Progress UI

Il main invia log speciali:

```
__SYNC_PROGRESS__:<percent>:<messaggio>
```

Il renderer aggiorna barra progresso senza inondare l'elenco attività.

Gestione errori

- `syncRunner` cattura errori, notifica desktop (se abilitate), rilancia messaggio friendly.
- Scheduler logga `Errore sincronizzazione: ...` senza crashare il processo.

7. Sistema aggiornamenti automatici

Basato su **electron-updater** e release **GitHub**.

Componenti

File	Ruolo
<code>updater.js</code>	Listener <code>autoUpdater</code> , emit eventi UI
<code>package.json</code> → <code>build.publish</code>	Provider GitHub
<code>dist/latest.yml</code>	Metadati versione (generato da electron-builder)

Comportamento app

- Init solo se `app.isPackaged` .
- Check automatico ~4s dopo avvio.
- Download e install **solo su azione utente**.
- Listener registrati una sola volta (`listenersRegistered`).

Release workflow (sintesi)

Vedi anche `[UPDATES.md] (../../UPDATES.md)` e §11.

1. Bump `version` in `package.json` .
2. Build piattaforma target.
3. Tag Git `vX.Y.Z` .
4. GitHub Release con asset: `.exe` / `.dmg` , `latest.yml` , `.blockmap` se presenti.
5. App installata legge `latest.yml` e propone update.

Versioning (calendar-based dal 2026)

Dalla `26.0.0` Easyfatt Sync adotta **calendar versioning**: MAJOR = anno (`26` = 2026, `27` = 2027...). Vedi `VERSIONING.md` .

- **MAJOR** — anno di riferimento; non implica automaticamente breaking change ma vengono comunque dichiarate in `RELEASE_NOTES.md` .
- **MINOR** — feature update durante l'anno (es. `26.1.0`).
- **PATCH** — bugfix (`26.0.1`).

Lo schema resta SemVer-compatibile per electron-updater: numeri MAJOR alti sono validi e ordinati correttamente.

8. Sistema supporto

Architettura

App Electron --POST JSON--> API Aven Labs --Resend--> Email interna + conferma cliente

- URL: `https://aven-labs.com/api/support/easyfatt-sync` (`supportConstants.js`).
- Timeout client: **30 secondi** (`support.js`).

Payload (campi principali)

Campo	Note
<code>name</code> , <code>email</code> , <code>message</code>	Obbligatori (validazione)
<code>phone</code>	Opzionale
<code>issueType</code>	Etichetta italiana da <code>supportIssueTypes.js</code>
<code>appVersion</code> , <code>platform</code> , <code>platformLabel</code>	Contesto tecnico
<code>lastSyncAt</code> , <code>lastSyncRows</code> , <code>googleAuthorized</code>	Diagnostica
<code>excelPath</code> , <code>sheetName</code>	Config (attenzione privacy in email interna)

Backend di esempio

Cartella: `backend-examples/nextjs/`

- Route: `app/api/support/easyfatt-sync/route.ts`

- Template email: `lib/support-email-templates.ts`
- Validazione server: `lib/support-validation.ts`

Variabili env tipiche:

```
RESEND_API_KEY=re_...  
SUPPORT_FROM_EMAIL="Aven Labs <support@aven-labs.com>"  
SUPPORT_TO_EMAIL="support@aven-labs.com"
```

Email cliente

Conferma automatica all'indirizzo inserito nel form, con footer privacy (link policy/termini).

9. Backup e ripristino

Formato file JSON

Estensione consigliata: `.easyfatt-sync-backup.json`

Nome manuale tipico: `easyfatt-sync-backup-YYYY-MM-DD.json`

Nome automatico: `easyfatt-sync-backup-YYYY-MM-DD-HH-mm.json`

Struttura payload

```
{  
  "app": "Easyfatt Sync",  
  "backupVersion": "1.0",  
  "backupType": "manual",  
  "createdAt": "2026-05-16T20:00:00.000Z",  
  "appVersion": "26.0.0",  
  "platform": "darwin",  
  "config": { },  
  "legalAccepted": true,  
  "legalAcceptedAt": "2026-01-01T00:00:00.000Z",  
  "legalVersion": "1.0",  
  "googleTokenIncluded": false,  
  "googleToken": null  
}
```

Campo	Descrizione
<code>backupType</code>	<code>"manual"</code>
<code>config</code>	Oggetto completo impostazioni electron-store
<code>googleToken</code>	Presente solo se l'utente ha spuntato l'inclusione

Non incluso: file Excel, dati clienti nel foglio, log applicativi.

Restore

1. Validazione `app` , `backupVersion` , `config` .
2. Merge config con default (`getDefaultConfig`).
3. Ripristino flag legali.
4. Token Google: scrittura o logout locale.
5. `restartScheduler` + `config-updated` verso UI.

Backup automatici

Impostazione	Default
<code>automaticBackupEnabled</code>	<code>false</code>
<code>automaticBackupFrequency</code>	<code>daily</code>
<code>automaticBackupTime</code>	<code>20:00</code>
<code>automaticBackupFolder</code>	<code>" "</code>
<code>automaticBackupRetention</code>	<code>10</code>
<code>automaticBackupIncludeGoogleToken</code>	<code>false</code>

- Settimanale: lunedì all'orario indicato.
- Mensile: primo giorno del mese.
- Cleanup: mantiene ultimi N file con pattern automatico; **non** cancella backup manuali.

10. Sicurezza

Electron hardening

In `main.js` → `webPreferences` :

```
contextIsolation: true,  
nodeIntegration: false,  
sandbox: true,  
preload: path.join(__dirname, "preload.js"),
```

Preload sicuro

- Nessun accesso diretto a `fs` , `process` , moduli Google.
- Solo metodi espliciti su `easyfattySync` .
- `open-external` valida URL (`^https?://`).

Token e credenziali

Asset	Posizione	Git
OAuth client secret	<code>oauth_credentials.json</code>	Ignorato
Refresh/access token	<code>EasyfattySync/token.json</code>	Ignorato
Config utente	<code>electron-store (userData)</code>	Locale
Diff snapshot Excel	<code>electron-store profileDataSnapshots</code>	Locale (mai esposti)
Diff details	<code>electron-store syncHistoryDiffs</code>	Locale (mai esposti)

Repository sicurezza (cose da non committare)

Oltre a `oauth_credentials.json` / `token.json` / `.env`, evita di committare:

- `dist/`, `*.exe`, `*.dmg`, `*.blockmap`
- `latest.yml`, `latest-mac.yml`
- `*.easyfatt-sync-backup.json`
- log applicativi (`*.log`, `logs/`)

Lista completa in `.gitignore` e `SECURITY.md`.

CI secret handling

Il workflow `release.yaml`:

1. Scrive `oauth_credentials.json` dal secret `OAUTH_CREDENTIALS_JSON` **solo durante la build**.
2. Esegue `rm -f oauth_credentials.json` in step `if: always()`.
3. Non logga mai il contenuto del secret (`printf '%s'` su file, senza `echo`).

Il `GITHUB_TOKEN` ha scope di repo limitato e viene revocato al termine del job.

API backend

- L'app non incorpora API key Resend.
- Il supporto passa da HTTPS verso dominio Aven Labs.

Buone pratiche release

- Firmare binari Windows/macOS quando possibile (SmartScreen / Gatekeeper).
- Non allegare token o Excel cliente alle issue GitHub.

11. Workflow release

GitHub Actions (release ufficiale)

File: `.github/workflows/release.yaml`.

Trigger: **push di un tag** `v*.*.*` (oppure `workflow_dispatch` manuale per build di prova).

Pipeline:

1. Checkout repo (`actions/checkout@v4`).
2. Setup Node **22** (`actions/setup-node@v4` , cache npm).
3. `npm ci` .
4. Crea `oauth_credentials.json` dal secret `OAUTH_CREDENTIALS_JSON` .
5. Build per OS della matrice:
 - `windows-latest` → `npm run dist:win` (NSIS x64).
 - `macos-latest` → `npm run dist:mac` (DMG ARM64, `CSC_IDENTITY_AUTO_DISCOVERY=false`).
6. Cleanup `oauth_credentials.json` (anche su failure).
7. Upload asset alla GitHub Release del tag (`softprops/action-gh-release@v2`):
 - `dist/*.exe` , `dist/*.dmg` , `dist/*.yaml` , `dist/*.blockmap` .

Secret richiesti (Settings → Secrets and variables → Actions):

Secret	Valore	Note
<code>OAUTH_CREDENTIALS_JSON</code>	JSON Desktop OAuth client Aven Labs	Iniettato solo durante la build, mai persistito
<code>GITHUB_TOKEN</code>	(auto)	Fornito da GitHub Actions, usato per Release

Checklist pre-tag

- ☐ `package.json` → `version` aggiornata (es. `26.0.0`)
- ☐ `RELEASE_NOTES.md` aggiornato con novità della release
- ☐ `UPDATES.md` e `VERSIONING.md` allineati
- ☐ `build.publish` corretto (`DanielMatei0/easyfatt-sync`)
- ☐ `npm run check` + `npm run security:check` puliti



Nessun file sensibile in `git status`



CI ha completato una build di prova (workflow_dispatch)



`npm run dev` funzionante in locale

Comandi tag/release

```
# scelta A - manuale
git add .
git commit -m "release: prepare Easyfatt Sync 26.0.0"
git push origin main
git tag v26.0.0
git push origin v26.0.0

# scelta B - npm version (crea commit + tag in un colpo)
npm version 26.0.0
git push origin main --follow-tags
```

Release manuale (fallback)

Sulla macchina target:

```
npm install
# oauth_credentials.json deve esistere in root
npm run dist:win      # su Windows o CI Windows
npm run dist:mac      # su Mac ARM
npm run dist:mac:intel
```

Controllare artefatti in `dist/` :

- Windows: `Easyfatt-Sync-Windows.exe` , `latest.yml` , `Easyfatt-Sync-Windows.exe.blockmap`
- macOS: `Easyfatt-Sync-macOS-arm64.dmg` , `latest-mac.yml` , `Easyfatt-Sync-macOS-arm64.dmg.blockmap`

Dettaglio nomi stabili: [release-assets.md](#).

Caricare manualmente tutti gli asset sulla GitHub Release (incluso `latest*.yml`).

Publish locale con token (raro)

```
export GH_TOKEN=<github_pat_with_repo_scope>
npm run dist:win
# electron-builder può pubblicare se configurato --publish always
```

In CI il publish è gestito dal workflow → preferire sempre GitHub Actions.

Compatibilità aggiornamenti

- Gli utenti su versione N ricevono update solo se `latest.yml` sulla release è coerente con la piattaforma installata.
- Non forzare downgrade: electron-updater installa versioni più recenti.
- Lo schema `26.x.x` è SemVer-compatibile: un utente su `1.0.3` riceve correttamente l'update a `26.0.0`.

12. TODO / roadmap

Roadmap indicativa (non impegnativa):

Priorità	Funzione	Note
Alta	Sync bidirezionale	Sheets → Excel o merge bidirezionale con conflict resolution
Media	Multi-sheet	Più fogli Excel / più tab Sheets
Media	Sync prodotti / fatture	Estensione modello dati oltre anagrafica clienti
Bassa	Cloud dashboard	Stato sync centralizzato per rivenditori
Bassa	Analytics	Metriche sync, errori aggregati (rispetto privacy)
Bassa	Multi-utente	Account Aven Labs, config cloud, ruoli

Ogni voce richiede ADR, aggiornamento `backupVersion` se cambia schema, e revisione privacy.

13. Troubleshooting tecnico

OAuth

Sintomo	Causa probabile	Azione
<code>oauth_credentials.json</code> mancante	File non in root dev	Copiare credenziali Desktop da GCP
Redirect error	Tipo client errato	Usare Desktop app, non Web senza URI
<code>invalid_grant</code>	Token revocato/ scaduto	Eliminare <code>token.json</code> , ricollegare
Porta occupata	Server OAuth precedente	Riavviare app, chiudere processi Electron

Sync

Sintomo	Causa probabile	Azione
File in uso	Easyfatt tiene lock	Chiudere export; verificare retry in <code>sync.js</code>
0 righe	Foglio Excel vuoto o solo intestazione	Verificare export Easyfatt
Permission denied Sheets	Account sbagliato / ID errato	Controllare <code>spreadsheetId</code> e permessi foglio
Doppia sync	Race (raro post-mutex)	Verificare un solo job cron per orario

Updater

Sintomo	Causa probabile	Azione
"Solo versione installata"	<code>npm run dev</code>	Testare su build packaged
Update non trovato	<code>latest.yml</code> mancante in release	Allegare YAML alla GitHub Release
Download fallito	Rete / proxy aziendale	Log <code>electron-log</code> , test manuale URL release

Build

Sintomo	Piattaforma	Azione
NSIS fallisce	Windows	Path senza spazi, antivirus disabilitato per <code>dist/</code>
DMG non si apre	macOS	Gatekeeper: firmare e notarizzare per distribuzione esterna
Icona mancante	Tutte	Verificare <code>assets/icon.png</code>

Permessi file

- Backup automatico: cartella deve essere scrivibile (`fs.accessSync W_OK`).
- Excel su rete SMB: latenza alta → aumentare debounce o disabilitare watch.

Token Google in backup

- Backup con token è sensibile: trattare file JSON come segreto.
- Dopo restore su altro PC, verificare che il refresh funzioni o ricollegare.

Contatti tecnici

- Supporto prodotto: support@aven-labs.com
 - Documentazione utente: `[docs/client/README.md](../client/README.md)`
-

*Ultimo aggiornamento documentazione: allineata a Easyfatt Sync **26.0.0** — Maggio 2026 (stack Electron 42, Node 22 in CI, backup v1.0).*